

Kristian David Castrillón Paz - Taller Gemini Final - Master IA y Data Science

El núcleo de este sistema es la integración segura de la API de Google Gemini, centralizando la API Key en el backend para proteger la cuota y evitar su exposición. Se implementó una lógica de Inferencia Adaptativa configurada específicamente para modelos de última generación (Gemini 2.5 Flash), garantizando una comunicación robusta y eficiente entre el servidor local y los servicios de inteligencia artificial de Google.

Link del vídeo demostrativo funcionando:

<https://drive.google.com/file/d/11l0QqVRD-mWST3qwPxC9HtqtYeubsArC/view?usp=sharing>

Anexos del código:

```
1 import os
2 import time
3 import uuid
4 import json
5 import requests
6 from datetime import datetime, timezone
7 from flask import Flask, request, jsonify, render_template
8
9 app = Flask(__name__)
10
11
12 # =====
13 API_KEY = "AIzaSyBQL35KuXIKXsqSP53dy16hdSyia-5ny[ENCRYPTION_KEY]"
14 DB_FILE = "data_store.json"
15
16 # MODO IA ACTIVADO
17 OFFLINE_MODE = False
18 USE_SMART_TITLES = False
19
20
21 MODEL_ENDPOINT = "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent"
22 sesiones_global = {}
23
```

Explicación:

Línea 13 (API_KEY): Es la llave maestra que permitió hablar con Google.

Línea 17 (OFFLINE_MODE = False): Es el interruptor de vida del sistema. En **False**, la IA está activa; si Google falla, se cambia a **True** para activar el simulador (Circuit Breaker) y que el programa no se detenga.

Línea 21 (MODEL_ENDPOINT): Es la dirección exacta del servidor de Google. Usamos **Gemini 2.5 Flash** porque descubrimos que era el único modelo que la cuenta tenía autorizado y que no daba error 404.

Línea 14 y 22 (DB y Sesiones): Definen que todo se guardará en un archivo JSON local y que, mientras el programa corre, las sesiones viven en un diccionario de Python para acceso ultra rápido ($O(1)$).

```

28 def cargar_desde_disco():
29     global sesiones_global
30     if os.path.exists(DB_FILE):
31         with open(DB_FILE, "r", encoding="utf-8") as f:
32             sesiones_global = json.load(f)
33     else:
34         # Seeding determinista (Al menos 5 ejemplos exigidos por protocolo)
35         temas = [
36             ("Cálculo: Integrales", False),
37             ("Setup RTX 3050", False),
38             ("Café Premium Cali", True),
39             ("Algoritmos O(n log n)", False),
40             ("Arquitectura REST", True)
41         ]
42         base_t = datetime(2026, 4, 20, 12, 0, 0, tzinfo=timezone.utc).timestamp()
43         for i, (tit, arch) in enumerate(temas):
44             s_id = f"sess_{uuid.uuid4().hex[:8]}"
45             t_iso = datetime.fromtimestamp(base_t + (i * 1800), timezone.utc).isoformat()
46             sesiones_global[s_id] = {
47                 "titulo": tit, "updated_at": t_iso, "archived": arch,
48                 "mensajes": [{"id": f"m_{uuid.uuid4().hex[:6]}", "role": "model", "parts": [{"text": f"Contexto {tit} inicializado."}], "timestamp": t_iso}]
49             }
50         guardar_en_disco()
51

```

seeding de los 5 ejemplos

```

108 def procesar_mensaje():
109     req = request.get_json()
110     id_s = req.get("session_id")
111     texto = req.get("text", "").strip()
112     audio_b64 = req.get("audio_b64")
113
114     if id_s not in sesiones_global: return jsonify({"err": "Sesión inválida"}), 400
115     if not texto and not audio_b64: return jsonify({"err": "Payload vacío"}), 400
116
117     t_now = datetime.now(timezone.utc).isoformat()
118     if len(sesiones_global[id_s]['mensajes']) == 0 and texto:
119         sesiones_global[id_s]['titulo'] = generar_titulo_ia(texto)
120
121     u_parts = []
122     if texto: u_parts.append({"text": texto})
123     if audio_b64: u_parts.append({"inlineData": {"mimeType": "audio/webm", "data": audio_b64}})
124
125     msg_u = {"id": f"m_{uuid.uuid4().hex[:6]}", "role": "user", "parts": u_parts, "timestamp": t_now}
126     sesiones_global[id_s]['mensajes'].append(msg_u)

```

Esta sección del código demuestra el cumplimiento del principio de **responsabilidad única**, donde la función se encarga estrictamente de validar, estructurar y temporalizar la información entrante antes de cualquier interacción con la API externa o la base de datos local.

static/[chat.js](#)

```

// --- LÓGICA DE INFERENCIA CON BLOQUEO ANTI-SPAM ---
const sendMessage = async (text, audioB64 = null) => {
    // Candado estricto: Si el botón ya está deshabilitado, aborta (previene spam de Enter)
    if (ui.send.disabled) return;

    if (!text.trim() && !audioB64) return;
    if (!state.currentId) {
        alert("Selecciona o crea una sesión primero.");
        return;
    }

    // Bloquear UI y limpiar input
    ui.send.disabled = true;
    ui.input.value = '';

```

```

},

// Listener para concurrencia multi-pestaña usando LocalStorage
window.onstorage = (e) => {
  if (e.key === 'v_sync') {
    loadSessions().then(() => {
      if (state.currentId && state.sessions.find(s => s.id === state.currentId)) {
        selectSession(state.currentId);
      } else {
        // Si la sesión activa fue purgada en otra pestaña, limpiar el lienzo
        state.currentId = null;
        ui.cWindow.innerHTML = '';
        ui.title.innerText = "Selecciona una sesión";
        ui.inputArea.classList.add('hidden');
      }
    })
  }
}

```

Bloqueo Anti-Spam: Implementa un candado lógico que deshabilita la interfaz durante la inferencia. Esto evita el desbordamiento de peticiones al backend y protege la cuota de la API.

Gestión de Concurrencia: El listener `window.onstorage` sincroniza el estado entre múltiples pestañas. Si se purga o archiva una sesión en una instancia, las demás se actualizan automáticamente, garantizando la integridad de la "verdad de los datos" en todo el navegador.

Validación Multimodal: Controla que el payload contenga texto o audio Base64 válido antes de la transmisión, reduciendo errores de red y tráfico innecesario.